CS-387L: Parallel & Distributed Computing Lab

ASSIGNMENT NO 1

NAME	Roll No
M Bilal	231400038

Question 1

Part(a)

Explanation

```
int N = 1024;

float h_infection[N];
float h_rate[N];
float h_projected[N];
```

The program creates 3 arrays on CPU:

- **h_infection[]** == Current infected people in each city
- **h_rate[]** == Infection spread rate for each city
- **h_projected[]** == Final projected infections

Simple Meaning:
CPU stores all initial data.

```
for (int i = 0; i < N; i++)
{
    h_infection[i] = (float)(i + 1);
    h_rate[i] = 1.0f + (i % 5) * 0.1f;
}
```

A loop fills infection and rate values:

Example:

- City 0 == Infection = 1, Rate = 1.0
- City 1== Infection = 2, Rate = 1.1
- City 2== Infection = 3, Rate = 1.2

```
cudaMalloc((void**)&d_infection, N * sizeof(float));
cudaMalloc((void**)&d_rate, N * sizeof(float));
cudaMalloc((void**)&d_projected, N * sizeof(float));
```

CPU asks GPU to reserve memory for data.

```
cudaMemcpy(d_infection, h_infection, N * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_rate, h_rate, N * sizeof(float), cudaMemcpyHostToDevice);
```

Input data is transferred from CPU to GPU.

```
int blockSize = 256;
int numBlocks = (N + blockSize - 1) / blockSize;
```

1024 cities are divided into multiple GPU threads for fast parallel execution.

```
pandemicKernel<<<numBlocks, blockSize>>>(d_infection, d_rate, d_projected, N);
```

Kernel launch

```
for (int i = 0; i < 10; i++)
{
    printf("City %d: Infections = %.2f, Rate = %.2f, Projected = %.2f\n",
           i, h_infection[i], h_rate[i], h_projected[i]);
}
```

Sample Output:

- City 0 == 1 × 1.0 = 1
- City 1 == 2 × 1.1 = 2.2
- City 2 == 3 × 1.2 = 3.6

```
cudaMemcpy(h_projected, d_projected, N * sizeof(float), cudaMemcpyDeviceToHost);
```

This line is used in CUDA to copy data from GPU back to CPU

```
__global__ void pandemicKernel(float *infection, float *rate, float *projected,
 int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N)
    {
        projected[i] = infection[i] * rate[i];
    }
}
```

This defines a CUDA kernel function that runs on the **GPU**
Each GPU thread calculates its **unique index (i)** so it knows which array element to process.
This checks that the thread index is valid (within array size) to avoid errors.
Each thread multiplies infection value with rate and stores result in projected[i].

```
    cudaFree(d_infection);
    cudaFree(d_rate);
    cudaFree(d_projected);

    return 0;
}
```

frees GPU memory at the end.

OUTPUT

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be

!./task1

City 0 | Infection = 1 | Spread Rate = 1 | Projected = 1
City 1 | Infection = 2 | Spread Rate = 1.1 | Projected = 2.2
City 2 | Infection = 3 | Spread Rate = 1.2 | Projected = 3.6
City 3 | Infection = 4 | Spread Rate = 1.3 | Projected = 5.2
City 4 | Infection = 5 | Spread Rate = 1.4 | Projected = 7
City 5 | Infection = 6 | Spread Rate = 1 | Projected = 6
City 6 | Infection = 7 | Spread Rate = 1.1 | Projected = 7.7
City 7 | Infection = 8 | Spread Rate = 1.2 | Projected = 9.6
City 8 | Infection = 9 | Spread Rate = 1.3 | Projected = 11.7
City 9 | Infection = 10 | Spread Rate = 1.4 | Projected = 14
```

Part(b)

If **N = 1,000,003** is not divisible by **256**, extra threads are created in the last block. Without a boundary check, these threads may access invalid memory and cause errors. To avoid this, use **if (i < N)** inside the kernel. The general formula for blocks is **(N + blockSize - 1) / blockSize**.

QUESTION 2

Part(a)

Explanation

```
signal_boost.cu
1    #include <stdio.h>
```

Used for printf() to display output

```
#include <math.h>
```

Used for mathematical functions

```
#include <cuda.h>
```

Used for CUDA GPU programming functions.

```
#include <cuda.h>

__global__ void signalKernel(float *A, float *B, float *combined, float *boosted, int N)
{
```

This is the GPU function. __global__ means CPU calls it, but it runs on GPU.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

Each GPU thread gets a unique index i to process one signal sample.

```
float h_A[N];
float h_B[N];
float h_combined[N];
float h_boosted[N];
```

CPU arrays for:

- h_A == Signal A
- h_B == Signal B
- h_combined == Combined signal
- h_boosted == Amplified signal

```
cudaMalloc((void**)&d_A, N * sizeof(float));
cudaMalloc((void**)&d_B, N * sizeof(float));
cudaMalloc((void**)&d_combined, N * sizeof(float));
cudaMalloc((void**)&d_boosted, N * sizeof(float));
```

Allocates GPU memory

```
cudaMemcpy(h_combined, d_combined, N * sizeof(float), cudaMemcpyDeviceToHost);
cudaMemcpy(h_boosted, d_boosted, N * sizeof(float), cudaMemcpyDeviceToHost);
```

This copies the **boosted signal data from GPU to CPU**.

```
int blockSize = 128;
int numBlocks = (N + blockSize - 1) / blockSize;
```

Calculates required blocks for all samples.

```
signalKernel<<<numBlocks, blockSize>>>(d_A, d_B, d_combined, d_boosted, N);
```

Runs GPU kernel for parallel signal processing.

```
for (int i = 0; i < 5; i++)
{
    printf("Sample %d: A=%.4f B=%.4f Combined=%.4f Boosted=%.4f\n",
           i, h_A[i], h_B[i], h_combined[i], h_boosted[i]);
}

cudaFree(d_A);
```

This is for output

Sample 0: A=0.0000 B=1.0000 Combined=1.0000 Boosted=2.5000
Sample 1: A=0.0100 B=0.9999 Combined=1.0099 Boosted=2.5248

```
__global__ void signalKernel(float *A, float *B, float *combined, float *boosted, int N)
{
```

This defines a CUDA kernel function that runs on the GPU. It takes two input arrays (A, B), and produces two output arrays (combined, boosted). N is the total number of elements.

```
    int i = blockIdx.x * blockDim.x + threadIdx.x;
```

This calculates the unique index for each GPU thread so every thread works on a different array element.

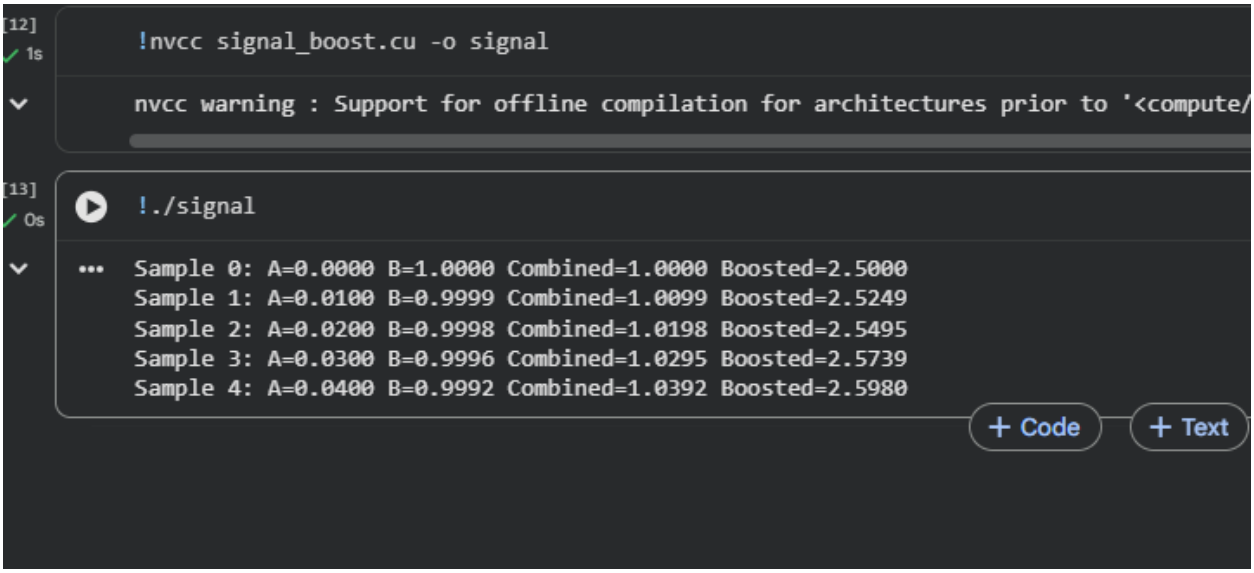
```
    if (i < N)
    {
        combined[i] = A[i] + B[i];
        boosted[i] = combined[i] * 2.5;
    }
}
```

Let's take a small example:

- A = [1.0, 2.0, 3.0]
- B = [0.5, 1.5, 2.5]
- N = 3

```
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_combined);
cudaFree(d_boosted);
```

Releases GPU memory.



Part(b)

A **single-kernel approach** is faster because both operations are performed in one kernel launch, reducing overhead. Using **multiple kernels** for a small task can increase execution time due to extra kernel launch and memory access overhead. Two separate kernels are more useful when each step needs separate debugging or when intermediate results are required. For the **Mars Rover signal booster**, I recommend the **single-kernel approach** because it is simpler and more efficient for this small processing pipeline.